

 **daniel-endraws** release lab3

1dc87d2 · last year 

Name	Name	Last commit date
..		
figs	release lab2	last year
sim	release lab2	last year
src	release lab2	last year
Makefile	release lab2	last year
README.md	release lab3	last year

README.md 

FPGA Lab 2: Introduction to FPGA Development

Prof. John Wawrzynek

TAs: (ordered by section) Daniel Endraws, Dhruv Vaish, Rohit Kanagal

Department of Electrical Engineering and Computer Science

College of Engineering, University of California, Berkeley

Before You Begin

Make sure that you have gone through and completed the steps involved in Lab 1. Let the TA know if you are not signed up for this class on Ed or if you do not have a class account (eecs151-xxx), so we can get that sorted out.

To fetch the skeleton files for this lab, `cd` to the git repository (`fpga-labs-<username>`) that you had cloned in the first lab and pull from the staff repository: `git pull staf main`

You should be comfortable with basic Verilog to complete this lab, be sure to refresh with and refer back to lecture slides or the [Verilog Primer Slides](#). *Note the verilog primer slides do not adhere to the no flip-flop inference policy outlined below*

Put yourself on the help queue if you have questions during the FPGA lab session or an FPGA TA's office hours. The queue form can be found on the EECS151 course website.

fpga_labs_sp24 / lab2 /  [Top](#)

Verilog. This policy applies to all labs and projects. You can find the file at the root of your lab repository or at `/home/ff/eecs151/verilog_lib/EECS151.v`.

- ▼ Table of Contents (click to close)
- [FPGA Lab 2: Introduction to FPGA Development](#)
 - [Before You Begin](#)
 - [A Structural and Behavioral Adder Design](#)
 - [Build a Structural 14-bit Adder](#)
 - [Makefile-Based Build Flow](#)
 - [Build a Behavioral 14-bit Adder](#)
 - [Simulating the Adder](#)
 - [Overview of Testbench Skeleton](#)
 - [Verilog Style Guide](#)
 - [Naming](#)
 - [Guidelines for Constructs \(excluding testbenches\)](#)
 - [Running the Simulation](#)
 - [VCS](#)
 - [Icarus Verilog](#)
 - [Analyzing the Waveform](#)
 - [Helpful Tip: Reloading Waveforms](#)
 - [Exhaustive Testing vs Random Testing](#)
 - [Build Your First Sequential Digital Circuit](#)

- [Clock Sources](#)
- [Build a 4-bit Counter](#)
- [Simulating the Counter](#)
 - [Analyzing the Simulation](#)
- [Put the Counter on the FPGA](#)
- [Lab Deliverables](#)
- [Acknowledgement](#)

A Structural and Behavioral Adder Design

Build a Structural 14-bit Adder

To help you with this task, you may wish to refer to the full adder slides and `Code Generation with for-generate loops` slide in these [Verilog Primer Slides](#) (slide 35).

Open `lab2/src/full_adder.v` and fill in the logic to produce the full adder outputs from the inputs. You can use either structural or behavior verilog for this.

Open `lab2/src/structural_adder.v` and construct a ripple carry adder using the full adder cells you designed earlier and a 'for-generate loop'. This must be in structural verilog.

Inspect the `z1top.v` top-level module and see how your structural adder is instantiated and hooked up to the top-level signals. For now, just look at the `user_adder` instance of your structural adder. As we learned in previous lab, the basic I/O options on the PYNQ-Z1 board are limited. *How are we getting two 3-bit integers as inputs to the adder from the PYNQ board?*

Makefile-Based Build Flow

Here is an overview of the `make` targets available in the `fpga_labs_sp23/labX` folders:

- `make lint` - Lint your Verilog with Verilator; checks for common Verilog typos, mistakes, and syntax errors
- `make elaborate` - Elaborate (but don't synthesize) the Verilog with Vivado and open the GUI to view the schematic
- `make synth` - Synthesize `z1top` and put logs and outputs in `build/synth`
- `make impl` - Implement (place and route) the design, generate the bitstream, and put logs and outputs in `build/impl`
- `make program` - Program the FPGA with the bitstream in `build/impl`
- `make program-force` - Program the FPGA with the bitstream in `build/impl` **without** re-running synthesis and implementation if the source Verilog has changed
- `make vivado` - Launch the Vivado GUI

Running `make program` will run all the steps required to regenerate a bitstream if the Verilog sources have changed.

Use this flow to generate a bitstream and program the FPGA. Try running `make lint` and `make elaborate` before you run `make program`.

Note: The errors and warnings that `make lint` and `make synth` will show are **incredibly helpful** in finding bugs. Things like combinatorial loops, multi-driven nets (where the same wire is trying to be set by competing sources), and bitwidth inconsistencies should be fixed! However in this lab `make lint` may give you a false warning about a combinational loop (`%Warning-UNOPTFLAT`) and might fail - you can safely ignore it. To see why click the following dropdown.

► Why is there a '%Warning-UNOPTFLAT'? (click to expand)

Try entering different binary numbers into your adder with the switches and buttons and see that the correct sum is displayed on the LEDs. If the circuit doesn't work properly on your first try, don't worry and move on to the next section where we simulate the `structural_adder` and you can easily fix bugs.

Build a Behavioral 14-bit Adder

Check out `lab2/src/behavioral_adder.v`. It has already been filled with the appropriate logic for you. Notice how behavioral Verilog allows you to describe the function of a circuit rather than the topology or implementation.

In `z1top.v`, you can see that the `structural_adder` and the `behavioral_adder` are both instantiated in the self-test section. A module called `adder_tester` (in `src/adder_tester.v`) has been written for you that will check that the sums generated by both your adders are equal for all possible input combinations of `a` and `b`.

If both your adders are operating identically, both RGB LEDs will light up (with red and blue). Verify this on your board.

Simulating the Adder

Let's run some simulations on the `structural_adder` in software to ensure it works. Typically, this is done before putting our design on the FPGA, but because the adder is simple, we started with the FPGA this time.

To do this, we will need to use a *Verilog testbench*. A Verilog testbench is designed to test a Verilog module by supplying it with the inputs it needs (stimulus signals) and testing whether the outputs of the module match what we expect.

Overview of Testbench Skeleton

Check the provided testbench skeleton in `lab2/sim/adder_testbench.v`. Let's go through what every line of this testbench does.

```
`timescale 1ns/1ns
```

The timescale declaration needs to be at the top of every testbench file.

```
`timescale (simulation time unit)/(simulation resolution)
```

The first argument to the timescale declaration is the simulation time unit. It specifies how much time *1 unit* of simulation time represents. In this case, we have defined the simulation time unit to be one nanosecond, so *1 unit* of simulation time equals one nanosecond.

The second argument to the timescale declaration is the simulation resolution, which tells the simulator to model transient behavior of your circuit at that resolution. In our example it is also 1 nanosecond. For this lab, we aren't modeling any gate delays, so the resolution can safely equal the time unit.

```
`define SECOND 1000000000
`define MS 1000000
```

These are some macros defined for our testbench. They are similar to `#define` directives in C. `define` macros should be used to declare named constants which are used in your RTL or testbench. For example, in this testbench, `SECOND` is defined as the number of nanoseconds in one second.

```
module adder_testbench();
    // Testbench code goes here
endmodule
```

`adder_testbench` is a testbench module. It is not intended to be placed on an FPGA, but rather it is to be run by a circuit simulator. All your testbench code goes in this module. We will instantiate our DUT (device under test) in this module.

```
reg [13:0] a = 0;
reg [13:0] b;
wire [14:0] sum;
```

Now we declare nets to hold the inputs and outputs that go in and out of the DUT. Notice that the inputs to the `structural_adder` are declared as `reg` type nets and the outputs are declared as `wire` type nets. The inputs nets (`a`, `b`) are declared as `reg` since they are assigned inside the testbench's `initial` block. The output net (`sum`) is declared as `wire` because *any connection to the output of an instantiated module must be a wire*.

Note: we can set the initial value of `reg` nets in the testbench to drive a particular value into the DUT at simulation time 0 (e.g. `a`).

Now we instantiate the DUT and connect its ports to the nets we have declared in our testbench:

```
structural_adder sa (
    .a(a),
    .b(b),
    .sum(sum)
);
```

The `initial begin ... end` block is the "main()" function for our testbench, and where the simulation begins execution.

This block enables simulator-specific waveform dumping for the 2 RTL simulators we are using. Verilog *system tasks* are prefixed by a dollar sign (e.g. `$dumpfile`). System tasks are similar to function calls, and they are implemented by the simulator.

In the `initial` block we drive the DUT inputs using blocking (`=`) assignments:

```
a = 14'd1;
b = 14'd1;
#(2);
assert(sum == 'd2);
```

In Verilog we can construct literal values using the syntax `<bit width>'<radix><value>`. Here are a few examples:

- `14'd1` = a 14-bit literal with value = 1 (`d` = radix 10 - decimal)
- `32'hcafe00d` = a 32-bit literal with value = 0xCAFEE00D (`h` = radix 16 - hexadecimal)
- `8'b1010_0001` = an 8-bit literal with value = 0b10100001 (`b` = radix 2 - binary)
 - *Note:* Use underscores to make reading the literal easier
- `'hFFFF_0000` = a 32-bit literal with value = 0xFFFF0000
 - *Note:* You can omit the bit width and let Verilog use the minimum number of bits necessary to construct the literal. This is not recommended for RTL, but is often OK for testbenches.

Since the adder is a combinational circuit, once we set its inputs (`a` and `b`), we **must advance time for the inputs to propagate to the output (`sum`) through the circuit**, before we can inspect the output.

We can advance simulation time using delay statements. A delay statement takes the form `#(units);`, where *1 unit* represents the simulation time unit defined in `timescale` declaration. For instance the statement `#(2);` would advance the simulation for 2 time units = $2 * 1\text{ns} = 2\text{ns}$.

After advancing time, `sum` should have the value 2. We can use the `assert` statement to check this condition. If the assertion fails, the simulator will emit an error:

```
assert(sum == 'd1') else $display("ERROR: Expected sum to be 1, actual value: %d", sum);
```



Note that you can provide an error message using the `$display` system task which takes a format string and arguments similar to `printf()` in C.

You can also use regular `if` statements to check conditions:

```
if (sum != 'd20) begin
    $error("Expected sum to be 20, a: %d, b: %d, actual value: %d", a, b, sum);
    $fatal(1);
end
```



The `$error()` system task also takes a format string and prints an error to the console. Both the `$error()` system task and `assert` won't halt and exit the simulation if there's a failure; you must use the `$fatal()` system task if you want to kill the simulation immediately.

Finally, the `$finish()` system task gracefully exits the simulation:

```
$finish();
end
```



Verilog Style Guide

We have established a few guidelines to write **synthesizable** high description languages (HDL) that is physically realizable. Because Verilog and SystemVerilog can be used for test harness design and modeling, there exist numerous constructs that **cannot be synthesized**. That is to say, these constructs cannot be mapped into actual hardware. There are also cases in which certain hardware descriptions can be translated into **extremely inefficient hardware**.

Naming

- **lower_snake_case** - declarations, instance names, signals (wires, regs, ports), variables, functions

```
module my_module ...
wire my_wire;
```



- **CAPS** - Constants, macros

```
localparam GENERATOR_CONSTANT = 1'b1;
`define FIXED_NUMBER 5
```



Guidelines for Constructs (excluding testbenches)

- **initial** - `initial` is synthesizable on FPGA, but will be ignored by most ASIC tools. Use of `initial` in hardware blocks typically signals improper reset/startup logic. Acceptable in testbenches.
- **delay(#)** - Delays cannot be realized in hardware. Acceptable in testbenches.

Running the Simulation

There are 2 RTL simulators we can use:

- **VCS** - proprietary, only available on lab machines, fast
- **Icarus Verilog** - open source, runs on Windows/OSX/Linux, somewhat slower

They all take in Verilog RTL and a Verilog testbench module and output:

- A waveform file (.vpd, .vcd, .fst) that plots each signal in the testbench and DUT across time
- A text dump containing anything that was printed during the testbench execution

VCS

We recommend using VCS. Run the Makefile from `fpga_labs_sp23/lab2`

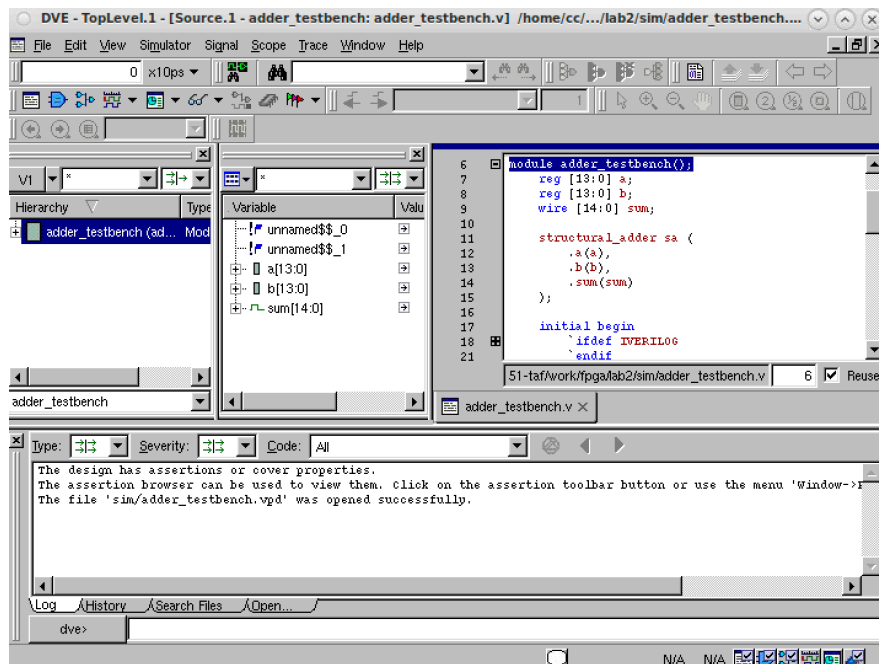
```
make sim/adder_testbench.vpd
```



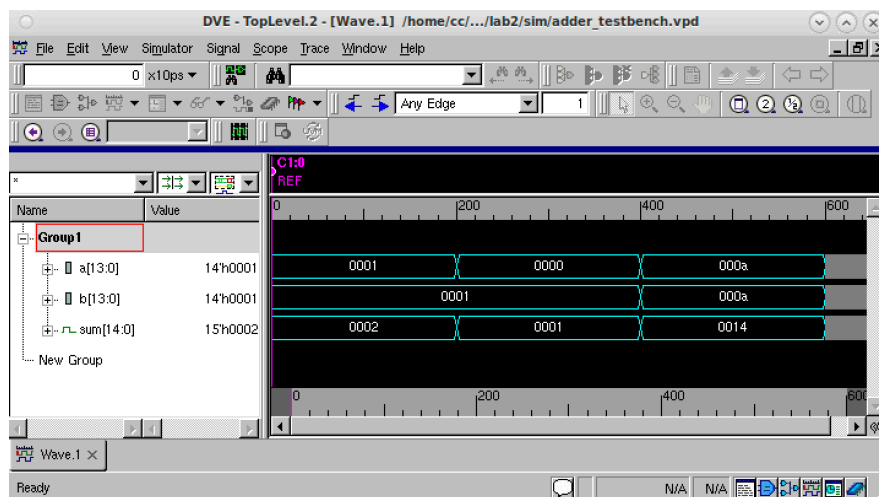
This will generate a waveform file `sim/adder_testbench.vpd`, which you can view using `dve`. Login to the lab machines physically or use X2go and run:

```
dve -vpd sim/adder_testbench.vpd &
```





From left to right, you can see the Hierarchy, Signals, and Source Code windows. The Hierarchy window lets you select a particular module instance in the testbench to view its signals. In the Signals window, you can select multiple signals (by Shift-clicking) and then right-click → 'Add To Waves' → 'New Wave View' to plot the waveforms for the selected signals.



Here are a few useful shortcuts:

- Click on waveform: Sets cursor position
- O: Zoom out of waveform
- +: Zoom into waveform
- F: Fit entire waveform into viewer (zoom full)
- Left Click + Drag Left/Right: Zoom in on waveform section

Icarus Verilog

Icarus Verilog is available on the lab machines. You can also install Icarus Verilog and gtkwave on your laptop via Homebrew ([iverilog](#), [gtkwave](#)) or a package manager with Linux or WSL.

Run `make sim/adder_testbench.fst` to launch a simulation with Icarus and to produce a FST waveform file. You can open the FST by running `gtkwave sim/adder_testbench.fst &` locally or on the lab machines.

Analyzing the Waveform

Open the waveform file using DVE or gtkwave. Plot the `a`, `b`, and `sum` signals. You should be able to see the signals change as specified in the testbench. For example, you should see the `a` signal start at 1 and then become 0 after 2 ns.

Helpful Tip: Reloading Waveforms

When you re-run your simulation and you want to plot the newly generated signals in DVE or gtkwave, you don't need to close and reopen the waveform viewer. Use `Shift + Ctrl + R` in gtkwave or `File → Reload Databases` in DVE to reload the waveform file.

Exhaustive Testing vs Random Testing

In the `adder_testbench` we have selected just a few values of `a`, `b`, and `sum` to verify. Is it reasonable to exhaustively test all possible combinations of `a` and `b` for a 14-bit adder in simulation? At what adder width is it no longer feasible?

You can use a `for` loop (similar to C) in the `initial` block to construct an exhaustive adder testbench:

```
integer ai, bi;
initial begin
    for (ai = 0; ai < 1024; ai = ai + 1) begin
        for (bi = 0; bi < 1024; bi = bi + 1) begin
            a = ai;
            b = bi;
            // delay + assert
        end
    end
end
```



When exhaustive testing isn't feasible, we often resort to random stimulus instead. In this case, you can use the `$urandom()` system task to generate an unsigned random number.

```
a = $urandom();
b = $urandom();
```



Build Your First Sequential Digital Circuit

In this section, you will design a 4-bit wrap-around counter that increments every second. The counter value is shown on the LEDs 0-3 of the PYNQ board.

Clock Sources

Look at the [PYNQ Reference Manual](#). Read Section 11 about the clock sources available on the PYNQ. We are using the 125 MHz clock from the Ethernet PHY IC on the PYNQ board that connects to pin H16 of the FPGA chip.

Look at the `lab2/src/z1top.v` top-level module and its `CLK_125MHZ_FPGA` input.

```
module z1top (
    input CLK_125MHZ_FPGA,
    ...
);
```



We can access the clock signal from our Verilog top-level module and can propagate this clock signal to any submodules that may need it.

In `z1top.v`, comment out line 2 to place the counter circuit on the FPGA, like this:

```
1 | // Comment out this line when you want to instantiate your counter
2 | // `define ADDER_CIRCUIT
```



Build a 4-bit Counter

Your circuit receives a clock signal with a period of 8 ns (125 MHz). How many clock cycles are equivalent to one second? Note:

Time (sec) = Clock Period * Number of cycles



Build a 4-bit counter that will increment its value every second (and **loop back to 0** once all 4 bits are used), and display the corresponding value on bits 3:0 of the IO LEDs. There is one caveat: the counter only counts if a 'clock enable' signal (in this case, called `ce`) is 1. **If it's 0, the counter should stay at the same value.**

Some initial code has been provided in `src/counter.v` to help you get started.

Note: It is helpful to think about what the circuit will look like before writing your verilog. Draw the modules / registers / logic and the wires that connect them (either in your head or on paper) before writing the HDL. Then after creating your module running `make elaborate` will open a gui showing what your HDL code connections actually look like.

Simulating the Counter

See the simulation skeleton in `sim/counter_testbench.v`.

The skeleton has code to generate a clock for the DUT (`counter`):

```
reg clock = 0;
always #(4) clock <= ~clock;
```



We set the value of the clock net to 0 at the very start of the simulation. The next line toggles the clock signal every 4ns, i.e. half period of 125 MHz clock.

Your task: write a testbench for the `counter`. Set the clock enable (`ce`) signal on and off and step time forward to test your counter. Use `assert` or `if` statements to verify the proper behavior of the `counter`.

You can use `repeat` and `@(event)` statements, in addition to `#(delay)`, to advance time based on a clock edge. For example

```
repeat (10) @(posedge c1ock);
```



will advance time until 10 rising clock edges are seen.

Run the simulation in the same way:

```
make sim/counter_testbench.vpd # vcs
make sim/counter_testbench.fst # iverilog
```



Note: simulating even a simple circuit for 1 second can take a long time. You may want to modify your `counter` to count up every 1/1000th of a second, so you can run simulation for a shorter duration. Once your design simulates successfully, you can change a constant in your `counter` to count up every second.

Analyzing the Simulation

When you open the waveform, you *may* see that your counter signal is just a red line. What's going on?

Fixing Unknown Signals

Blue lines (shown as `z`) in a waveform viewer indicate high-impedance (unconnected) signals. We won't be using high-impedance signals in our designs, so blue lines or `z` indicate something in our testbench or DUT isn't wired up properly.

Red lines (shown as `x`) in a waveform viewer indicate a signal has an unknown value. This can happen if you create registers with undefined initial values. However, the registers in the `EECS151.v` library declare initial values, so you shouldn't encounter issues with this. It is good practice to use a reset signal to force all registers to a known, desired initial state, but this isn't required for this lab.

Do not set initial values for 'wire' type nets; this will cause issues with synthesis, and may cause X's in simulation.

Now run the simulation again until you feel confident in your design.

Put the Counter on the FPGA

Once you're confident that your counter works, program the FPGA using the make-based flow as before. `z1top` connects your counter to the 125 MHz clock and connects switch 0 as the clock enable signal. **If you modified `src/counter.v`'s counting period to make the simulation shorter, make sure you undo that change when programming the FPGA (i.e. make sure it counts 1 second when programming).** If done correctly, LEDs 0 through 3 should continually count up by 1 each second.

This process, where we use simulation to verify the functionality of a module before programming it onto the FPGA, is invaluable and will be the one we use throughout this semester.

Lab Deliverables

Lab Checkoff (due: next lab)

To checkoff for this lab, have these things ready to show the TA:

- Your FPGA, programmed with the adder circuit and with both RGB LEDs (LEDs 4 and 5) lit up showing correctness. Be ready to explain how your structural adder works.
- A waveform of the testbench you wrote for your counter and its clock enable functionality.
- Be ready to program your FPGA with the counter and demonstrate that the counter can be enabled/disabled.

Acknowledgement

This lab is the result of the work of many EECS151/251 GSIs over the years including:

- Sp12: James Parker, Daiwei Li, Shaoyi Cheng
- Sp13: Shaoyi Cheng, Vincent Lee
- Fa14: Simon Scott, Ian Juch
- Fa15: James Martin
- Fa16: Vighnesh Iyer
- Fa17: George Alexandrov, Vighnesh Iyer, Nathan Narevsky
- Sp18: Arya Reais-Parsi, Taehwan Kim
- Fa18: Ali Moin, George Alexandrov, Andy Zhou
- Sp19: Christopher Yarp, Arya Reais-Parsi
- Fa19: Vighnesh Iyer, Rebekah Zhao, Ryan Kaveh
- Sp20: Tan Nguyen
- Fa20: Charles Hong, Kareem Ahmad, Zhenghan Lin
- Sp21: Sean Huang, Tan Nguyen
- Fa21: Vighnesh Iyer, Charles Hong, Zhenghan Lin, Alisha Menon

- Sp22: Alisha Menon, Yikuan Chen, Seah Kim
- Fa22: Simon Guo, Yikuan Chen
- Sp23: Rahul Kumar, Yukio Miyasaka, Dhruv Vaish
- Sp24: Daniel Endraws

EECS150 / fpga_labs_sp24 Public

<> Code Issues Pull requests Actions Projects Security Insights

fpga_labs_sp24 / lab3 /

 daniel-endraws minor lab updates ab4190a · last year

Name	Name	Last commit date
..		
figs	release lab3	last year
sim	release lab3	last year
src	release lab3	last year
Makefile	release lab3	last year
README.md	minor lab updates	last year

README.md

FPGA Lab 3: More Sequential Circuits, Audio "DAC"

Prof. John Wawryznec

TAs: (ordered by section) Daniel Endraws, Dhruv Vaish, Rohit Kanagal

Department of Electrical Engineering and Computer Science

College of Engineering, University of California, Berkeley

Overview

In this lab we will:

- Build input conditioning circuits so we can safely use the buttons as inputs to sequential circuits
 - Write parameterized Verilog modules
 - Use fork/join simulation threading in Verilog testbenches
 - Test the button signal chain on the FPGA
- Create an audio "DAC" using a PWM generator
- Write a square wave signal generator
 - Test the circuit on the FPGA

Before You Begin

Lab Video Walkthrough

Previous staff have prepared a [video walkthrough](#) on how the Audio component of the lab works. This video will help you understand how we can generate sound on the FPGA and the idea behind the Digital-to-Analog Converter and Square Wave Generator that you will be writing. **We highly recommend watching it before attempting the audio portion of the lab.**

Verilog Reading

Look through these documents if you haven't already.

- [Verilog Primer Slides](#) - overview of the Verilog language
- [wire vs reg](#) - the differences between `wire` and `reg` nets and when to use each one
- [always @ blocks](#) - understanding the differences between the two types of `always @` blocks and what they synthesize to
 - *Note: This does not adhere to the no flip-flop inference policy enforced in this class, i.e. you should not use `always @(posedge clk)` blocks, although they are important to learn about!*

▼ Table of Contents (click to close)

- [FPGA Lab 3: More Sequential Circuits, Audio "DAC"](#)
 - [Overview](#)
 - [Before You Begin](#)
 - [Lab Video Walkthrough](#)
 - [Verilog Reading](#)

- [Additional Testbench Constructs](#)
- [Input Conditioning Circuits](#)
 - [Synchronizer](#)
 - [Edge Detector](#)
 - [Debouncer](#)
 - [Debouncer Implementation Notes](#)
 - [Debouncer Testbench Notes](#)
 - [Button Parser on the FPGA](#)
 - [Counter with Button Parser](#)
 - [Counter Testbench](#)
 - [Play and Pause your Counter](#)
- [PWM](#)
 - [Audio Out](#)
 - [PWM Generator as DAC](#)
 - [DAC Testbench](#)
 - [Square Wave Generator](#)
 - [Square Wave Generator Testbench](#)
 - [On the FPGA](#)
- [Lab Deliverables](#)
- [Acknowledgement](#)

Additional Testbench Constructs

The testbenches provided in this lab use some constructs we haven't covered yet. Here is an overview of constructs you should know:

- `@(posedge <signal>)` and `@(negedge <signal>)`

- These are event trigger statements which are similar to the delay statements you've seen before
- `#10` advances the simulation by 10 timesteps
- `@(posedge <signal>) / @(negedge <signal>)` will advance the simulation until the `<signal>` rises/falls.
- For example, this block will advance simulation time until we have seen two rising edges of `clock` :

```
@(posedge clock);
@(posedge clock);
```



- `repeat`

- Acts like a `for` loop but without an increment variable
- For example, this block will advance the simulation until we have seen 2 falling clock edges, and will then advance further until we have seen 10 rising clock edges.

```
repeat (2) @(negedge clk);
repeat (10) begin
    @(posedge clk);
    $display("Another statement here");
end
```



- `$display`

- A print statement. Similar to `printf` in C. You call `$display()` with a format string and arguments.

```
$display("Wire x in decimal is %d", x);
$display("Wire x in binary is %b", x);
// You can even print out the current simulation time
$display("Time %d", $time);
```



- Lab 2 covered `$error()` which works like `$display()` , but is used to print an error message

- `Tasks`

- Tasks are subroutines that can group together simulation commands (similar to functions).
- They can take inputs and assign outputs. They can be called like functions in an `initial` block.

```
task wait_for_n_clocks();
input [7:0] num_edges;
begin
    repeat (num_edges) @(posedge clk);
end
endtask

initial begin
    a = 16'd1;
    b = 16'd3;
    wait_for_n_clocks(10);
    assert(sum == 16'd4);
end
```



- `fork / join`

- Allows you to execute blocks simulation code in parallel. You create a fork block with the keyword `fork` and end the block with the keyword `join`.

```
fork
begin
    task1();
end
begin
    $display("Another thread");
    task2();
end
join
```



- Multiple threads of execution are created by putting multiple begin/end blocks in the fork-join block.
- In this example, thread 1 runs `task1()`, while thread 2 first `$display`s some text then runs `task2()`. The threads run in parallel.

- Hierarchical Paths

- You can access signals *inside* an instantiated module for debugging purposes using `.` to provide a path to the signal. This can be helpful in some cases where you want to look at an internal signal but don't want to create another output port just for debug.

```
// RTL
module m(input [1:0] a, output y);
    wire q; // internal signal
    assign q = |a;
    assign y = ~q;
endmodule
```



```
// Testbench
m dut(.a(a), .y(y));
$display("a: %d", a);
$display("dut.q: %d", dut.q);
```



Input Conditioning Circuits

We want to use the buttons as inputs to a sequential circuit. For example, we may want to press a button to increment a counter, and press another one to decrement a counter.

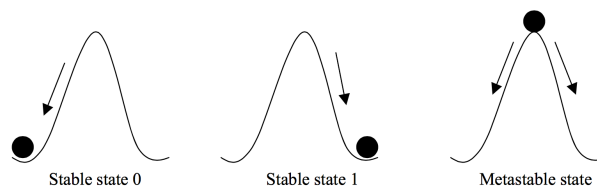
However, there are a few issues with using the raw button signals in our circuits:

- A button press is an asynchronous signal (it is not aligned to any clock edge): we need to sample the button signal safely from our 125 MHz clock domain
- When we press a button, we want a signal that goes high for only one clock cycle indicating a single press: we need to detect the rising edge of the button signal
- When a button is pressed, the connection of the button net to VDD isn't instant and clean (the button *bounces*): we need to add hysteresis

Synchronizer

In synthesizable Verilog (RTL), digital signals are either 0's or 1's (low/high voltage level). If the circuit is well designed and timed (fully synchronous), we only have to worry about the low and high voltage states.

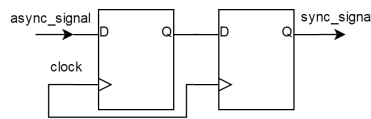
The signals coming from the push buttons and slide switches on the PYNQ board don't have an associated clock (they are asynchronous). When the button signals are put through a register, its hold or setup time requirements may be violated. This may put that register into a *metastable* state.



The 'ball on a hill' metaphor for metastability. If a register's timing constraints are violated, its output voltage oscillates and after some time unpredictably settles to a stable state.

In a fully synchronous circuit, the timing tools will determine the fastest clock frequency under which the setup time constraints are all respected and the routing tools will ensure that any hold time constraints are handled. An asynchronous signal could violate timing constraints, make the output of the register metastable, and violate more downstream constraints: making the circuit fail.

We will implement a synchronizer circuit that will safely bring an asynchronous signal into a clock domain. The synchronizer needs to have a



1-bit 2 Flip-Flop Synchronizer

For synchronizing one bit, it is a pair of flip-flops connected serially.

Edit `lab3/src/synchronizer.v` to implement the two flip-flop synchronizer. This module is parameterized by a `WIDTH` parameter which controls the number of one-bit signals to synchronize.

A testbench is provided in `lab3/sim/sync_tb.v`. **Run the testbench** with `make sim/sync_tb.vpd` (VCS) or `make sim/sync_tb.fst` (Icarus Verilog) as usual.

Edge Detector

We want to convert the low-to-high transition of a button press to a 1 clock cycle wide pulse that the rest of our design can use.

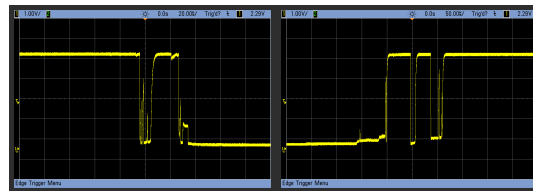
Implement a parameterized-width edge detector in `lab3/src/edge_detector.v`.

A testbench is provided in `lab3/sim/edge_detector_tb.v`. **Run it as usual.**

The edge detector testbench tests that your `edge_detector` outputs a 1 cycle wide pulse when its corresponding input transitions from 0 to 1. You should **visually inspect the waveform** to verify the correct behavior too.

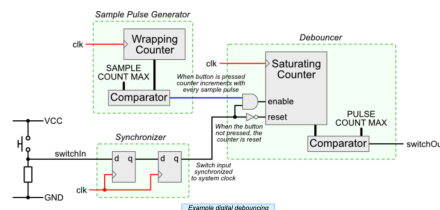
Debouncer

When we press or depress a button, the signal doesn't behave like a perfect step function. Instead the button signal is glitchy due to mechanical 'bounce'.



Scope waveform of button bounce when pressing and depressing a button.

If we naively used the button signal directly, there would be many spurious "button presses". The debouncer circuit takes a button's glitchy digital signal and outputs a clean high-to-low signal indicating a single button press.

Architecture of a simple debouncer circuit. [Source](#)

The debouncer module should consist of the following (*note the input to the debouncer is the output of the synchronizer module so you don't need to include it inside of your debouncer*):

- **Sample Pulse Generator** - Tells our saturating counter when to sample the input signal. It should output a 1, every `SAMPLE_CNT_MAX` clock cycles. By default `SAMPLE_CNT_MAX` is set to 65000.
- **Saturating Counter** - This is a counter that counts up to and including `PULSE_CNT_MAX` (and saturates).

The circuit implementation has the following behavior:

- All counters should start at 0.
- If the sample pulse generator emits a 1 on a clock edge, increment the saturating counter if the input signal is also 1, else reset the saturating counter to 0.
- Once the saturating counter reaches `PULSE_CNT_MAX`, it should hold that value indefinitely until the sampled input signal becomes 0.
- The `debounced_signal` of your debouncer should be an equality check between the saturating counter and `PULSE_CNT_MAX`. This `debounced_signal` should be a clean, stable button press signal.

At a high level, the debouncer ensures that you are only outputting a 1 if the glitchy signal has been held high for a given amount of samples (`PULSE_CNT_MAX`) where each sample occurs every `SAMPLE_CNT_MAX` cycles.

Start by thinking about how to implement the debouncer for `WIDTH=1` and then try to extend the logic to a parametrizable input width.

Implement the debouncer in `lab3/src/debouncer.v`. You can use the same sample pulse generator for all input signals into your debouncer, but you should have a separate saturating counter per input signal.

Debouncer Implementation Notes

A testbench is provided in `lab3/sim/debouncer_tb.v`. Make sure you **understand what the testbench is doing**. Run it as usual.

The debouncer testbench has 2 tests:

- Verifies that if a glitchy signal initially bounces and then stays high for *less* than the saturation time, that the debouncer output never goes high.
- Verifies that if a glitchy signal initially bounces and then stays high for *more* than the saturation time, that the debouncer goes high and stays high until the glitchy signal goes low.

Debouncer Testbench Notes

If you are seeing issues where certain registers are red lines (X's), make sure you give them an initial state and that they aren't being driven by different sources (multi-driven net).

Button Parser on the FPGA

Now that we have tested all the modules required to produce a clean, stable button press signal. It's time to put everything together.

Counter with Button Parser

Look at `src/button_parser.v` which combines the synchronizer, debouncer, and edge detector in a chain.

Then look at `src/counter.v` which implements a simple up/down counter controlled by button presses. **Observe** how these 2 modules are connected at the top-level in `src/z1top.v`.

Run the FPGA build flow to put the counter on the FPGA and verify it works as expected. You may get some `%Warning-WIDTH` errors in your debouncer from running `make lint`: these can be safely ignored if they involve the module parameters.

Look at `src/button_parser.v` which combines the synchronizer, debouncer, and edge detector in a chain.

For Windows Users: You may need to change `rtol` to `floor` instead in `src/z1top.v`

Counter Testbench

Write a testbench from scratch for `counter.v` in `sim/counter_tb.v`. Make sure you use asserts or `$error` statements that verify the correct functionality. Use the provided testbenches and your `counter_testbench` from lab2 as a guide.

Play and Pause your Counter

Right now, your `counter` is in a static mode; you can only increment or decrement the counter value for each button press. **Modify** `counter.v` such that when you press `buttons[2]`, your counter starts incrementing every one second (running mode), and pauses when you press `BTN2` again (static mode).

You can incorporate the code you wrote from lab2. Use the `CYCLES_PER_SECOND` parameter in `counter.v` to set the threshold for your cycle counter.

Extend `sim/counter_tb.v` to test changing between running and static mode. In your testbench, you can **override** `CYCLES_PER_SECOND` when instantiating your `counter` to make simulation fast.

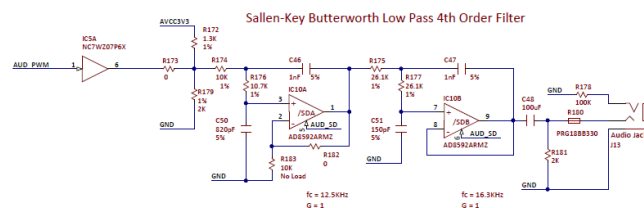
We'd like suggest that you **think about the structure of your design ahead of coding**. The debouncer circuit figure above is a great example. Remember that Verilog coding is all about describing your hardware circuit (very unlike previous imperative programming languages you may be used to)! Sketch a block diagram of your mode counter circuit in terms of the `button_parser` block and some register blocks from `EECS151.v` as you see fit. Label the relevant input and output signals. Feel free to use any logic gates, MUXes, or adders. Don't worry about listing all the details.

PWM

Staff has put together a conceptual walkthrough for the DAC, and square wave generator: <https://youtu.be/1FiaAPFCMFU> (This is the same as the beginning of the lab, so you can skip if you have already watched it)

Audio Out

Look at Section 14 of the [Pynq Reference Manual](#) which describes the mono audio out feature on the Pynq board.



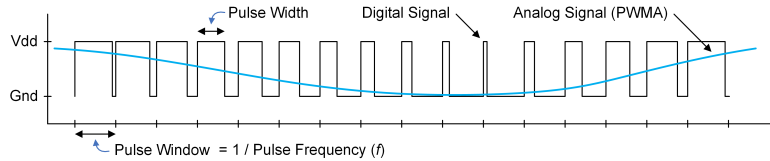
The FPGA pin R18 is connected to the `AUD_PWM` net. The FPGA can drive this net with a PWM signal which goes through a low-pass filter and is driven into the audio jack on the Pynq board.

There's also an `AUD_SD` net connected to FPGA pin ~~R17~~ ^{D12}, which turns off the opamps in the low-pass filter. Setting `AUD_SD` to 1 enables the audio output.

Find these signals in the `src/z1top.xdc` file, and note how they appear in the `src/z1top.v` port list.

PWM Generator as DAC

Since we have a fast 125 MHz clock, we can generate a PWM signal to drive the `AUD_PWM` net.



Let's make the pulse window **1024 cycles** of the 125 MHz clock. This gives us 10 bits of resolution, and gives a PWM frequency of $125\text{MHz} / 1024 = 122 \text{ kHz}$ which is much greater than the filter cutoff.

Implement the circuit in `src/dac.v` to drive the `pwm` output based on the `code` input. Assuming clock cycles are 0-indexed, the `code` is the clock cycle (up to and including) in the pulse window during which the `pwm` output should be high. `code = 0` is an edge case where `pwm` should be 0 for the entire pulse window of 1024 cycles.

For example:

- If `code = 0` :
 - `pwm` should be 0 for the entire pulse window of 1024 cycles
- If `code = 511` , `pwm` should be 1 for cycles 0 - 511 and 0 for cycles 512-1023
 - $511 - 0 + 1 = 512$ cycles high, $1023 - 512 + 1 = 512$ cycles high
- If `code = 1023` , `pwm` should be 1 the entire pulse window
 - $1023 - 0 + 1 = 1024$ cycles high

You can assume that `code` will only change every 1024 cycles.

The DAC should also output a signal called `next_sample` to tell the outside world that it can safely change the `code` before another pulse window begins. `next_sample` should be 1 on the final cycle of the pulse window.

DAC Testbench

Run the testbench in `sim/dac_tb.v` to verify this functionality - check the waveform too. *Note:* the testbench sets `CYCLES_PER_WINDOW` to 8 to make the simulation easier to debug.

Square Wave Generator

Now we have a DAC, but we need to give it samples to play. **Write a square wave sample generator** in `src/sq_wave_gen.v` which takes a 125 MHz clock and the `next_sample` signal from the DAC, and outputs the `code` for it to play.

`next_sample` indicates when the `sq_wave_gen` module should emit a new sample of the 440 Hz square wave. When `next_sample` is high, you should emit the next sample of the square wave on the `code` output on the next rising clock edge. When `next_sample` is low, you should freeze the state of your module since the outside world isn't requesting a new sample.

The square wave generator should output the codes for a 440 Hz square wave.

Note: $125\text{e}6 / 1024 / 440 / 2 = 138.7 \sim 139$

- $125\text{e}6$ clock cycles / second
- 1024 clock cycles / pulse window cycle
- 440 square cycles / second
- 2 value partitions / square cycles (a square wave is high half the time & low the other half)

When the square wave is high, the `code` should be 562, and when the square wave is low, the `code` should be 462. Avoid using the full `code` range from 0-1023 to keep the volume low.

Square Wave Generator Testbench

We have provided a simple testbench in `sim/sq_wave_gen_tb.v` which pulls about 1 second worth of samples ($125\text{e}6 / 1024 = 122070$) from your `sq_wave_gen` module and writes them to a file in `sim/codes.txt`.

You can run a script to convert this text file to an audio file: `cd lab3 && ../scripts/audio_from_sim sim/codes.txt`

Play the `output.wav` audio file by connecting your headphones to the machine and using `aplay output.wav` or by opening it with VLC. Additionally you can use `scp` to copy the file to your personal machine (`scp eecs151-xxxx@c111-xx.eecs.berkeley.edu: <path to src> <path to dst>`). Verify this sounds like a 440 Hz square wave; [use this website](#) for reference (be sure to change the wave type from a sin wave to a square wave). **Keep your volume low** - square waves are harsh.

On the FPGA

Look at `src/z1top.v` to see how the `dac` and `sq_wave_gen` are connected. Use the standard flow to **generate a bitstream**. Program the FPGA, plug headphones into the audio out port, and verify that you hear a 440 Hz tone without any glitching. Use `SWITCHES[1]` to toggle the sound on and off.

Lab Deliverables

Lab Checkoff (due: next lab)

To checkoff for this lab, have these things ready to show the TA:

- A counter testbench and its waveform
- Be ready to explain what order the button parser connects the input conditioning circuits
- A functioning button parser that you can use to control the counter in running and static mode
- A functioning DAC that can emit a fixed 440Hz square wave tone

Acknowledgement

This lab is the result of the work of many EECS151/251 GSIs over the years including:

- Sp12: James Parker, Daiwei Li, Shaoyi Cheng
- Sp13: Shaoyi Cheng, Vincent Lee
- Fa14: Simon Scott, Ian Juch
- Fa15: James Martin
- Fa16: Vighnesh Iyer
- Fa17: George Alexandrov, Vighnesh Iyer, Nathan Narevsky
- Sp18: Arya Reais-Parsi, Taehwan Kim
- Fa18: Ali Moin, George Alexandrov, Andy Zhou
- Sp19: Christopher Yarp, Arya Reais-Parsi
- Fa19: Vighnesh Iyer, Rebekah Zhao, Ryan Kaveh
- Sp20: Tan Nguyen
- Fa20: Charles Hong, Kareem Ahmad, Zhenghan Lin
- Sp21: Sean Huang, Tan Nguyen
- Fa21: Vighnesh Iyer, Charles Hong, Zhenghan Lin, Alisha Menon
- Sp22: Alisha Menon, Yikuan Chen, Seah Kim
- Fa22: Jennifer Zhou, Yikuan Chen, Simon Guo
- Sp23: Rahul Kumar, Yukio Miyasaka, Dhruv Vaish
- Sp24: Daniel Endraws

fpga_labs_sp24 / lab4 /

daniel-endraws minor lab updates

ab4190a · last year

Name	Name	Last commit date
..		
figs	release lab4	last year
scripts	release lab4	last year
sim	release lab4	last year
src	minor lab updates	last year
Makefile	release lab4	last year
README.md	minor lab updates	last year

README.md

FPGA Lab 4: Tunable Wave Generator, NCO, FSMs, RAMs

Prof. John Wawryznek

TAs: (ordered by section) Daniel Endraws, Dhruv Vaish, Rohit Kanagal

Department of Electrical Engineering and Computer Science

College of Engineering, University of California, Berkeley

Overview

In this lab we will:

- Extend the functionality of the square wave generator we built in lab3
 - Use the buttons to adjust the output wave frequency
- Write a Numerically Controlled Oscillator (NCO)
 - Initialize a ROM with a binary file and use it as an LUT
 - Design a phase accumulator (PA)
- Design an FSM
 - Use buttons to switch between states
 - Use the FSM to control the NCO using a RAM
 - Test the circuit on the FPGA

Before You Begin

Fetch Latest Lab Skeleton

```
cd fpga-labs-<username>
git pull staff main
```

If you face any divergent branch issues, merge the staff changes into your branch with:

```
git merge staff/main
```

Copy Sources From Previous Lab

```
cp lab3/src/synchronizer.v lab3/src/debouncer.v lab3/src/edge_detector.v lab4/src
```

Look through these documents if you haven't already.

- [Verilog Primer Slides](#) - Overview of the Verilog language
- [FSM](#) - Finite State Machines in Verilog (*Note does not adhere to no-flipflop inference policy)

[fpga_labs_sp24 / lab4 /](#)

- [Overview](#)
 - [Before You Begin](#)
- [Tunable Square Wave Generator](#)
 - [Implementation](#)
 - [Verification](#)
 - [FPGA](#)
- [Numerically Controlled Oscillator \(NCO\)](#)
 - [Overview](#)
 - [Implementation](#)
 - [Verification](#)
 - [FPGA](#)
- [FSM + Note Sequencer \(RAM\)](#)
 - [Sequencer RAM](#)
 - [FSM Specification](#)
 - [Testbench](#)
- [Put Everything Together](#)
- [Lab Deliverables](#)
- [Acknowledgement](#)

Tunable Square Wave Generator

In lab 3, we built a simple square wave generator which can emit a fixed 440Hz square wave tone. We would like to add more functionality.

Implementation

Support 2 modes of frequency adjustment which can be achieved by directly adjusting the period:

- *Linear Period Adjustment*: increase the frequency of the square wave linearly using the `STEP` parameter to determine how much to adjust the square wave period for every button press
 - For example, last lab we inverted the square wave every 139 samples, after an increase with a step of 10 we should now invert the square wave every 149 samples
- *Exponential Period Adjustment*: double or halve the period of the square wave for every button press (*hint*: use bitshifts)

Use the button inputs as follows:

- `button[0]` to increase the square wave frequency (decrease period)
- `button[1]` to decrease the square wave frequency (increase period)
- `button[2]` to switch between the 2 modes of frequency adjustment (linear period step/exponential)

Use `leds[0]` to display the frequency adjustment mode. The other `leds` can be set as you wish.

Since we now have a working button parser, we will use an explicit reset signal (`rst`) to make sure our registers don't hold undefined values (`x`) during simulation, and to gain the ability to reset our circuits at runtime.

When `rst` is high on a rising clock edge, you should be in:

- 440 Hz square wave frequency
- Linear incrementing mode

by resetting all registers in your circuit to known values.

Manually copy your DAC from `lab3/src/dac.v` to `lab4/src/dac.v`. **Use the new `rst` signal** to reset registers inside your DAC *instead of* using initial values. Example:

```
// Explicit global reset
input clk, rst;
wire [4:0] count, count_next;

REGISTER_R #(
    .N(5),
    .INIT(5'd0) // Reset value
) counter (.q(count), .d(count_next), .rst(rst), .clk(clk));

// ... (code here) ...
```



Use your solution from lab 3 to **implement the new square wave generator** in `src/sq_wave_gen.v`. You should support a square wave frequency range from 20 Hz to 10 kHz. (*hint*: calculate the corresponding period...).

Verification

Extend the testbench in `sim/sq_wave_gen_tb.v` to verify the reset and frequency adjustment functionality of your `sq_wave_gen`.

Make sure your RTL can **handle overflow** (what happens when you keep pressing the same button?)

The testbench has 2 simulation threads

- The first one pulls samples from the `sq_wave_gen` at random intervals
- The second one is for you **to write by simulating button presses**. You should use the `num_samples_fetched` variable to advance time in the simulation.
 - *Note:* the sample rate is $125\text{e6} / 1024 = 122 \text{ kHz}$
 - *Example:* to wait for a quarter of a second, you can use the delay statement `@(num_samples_fetched == 30517)`
 - This is computed by `wait_time * sample_rate = 0.25 * (125e6 / 1024) = 30517`

You can use the same script from lab 3 to convert the simulation output to an audio file.

```
../scripts/audio_from_sim sim/codes.txt
aplay output.wav
```



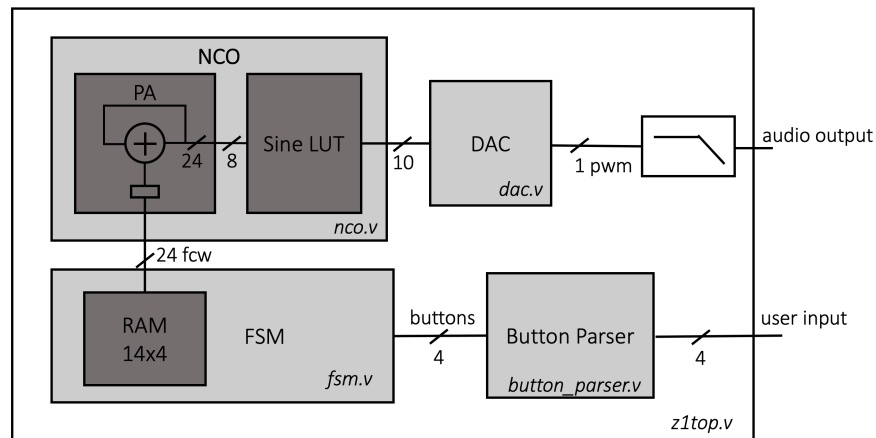
FPGA

Look at `src/z1top.v` to see how the new `sq_wave_gen` is connected. Use `SWITCHES[1]` to turn the audio output on/off, and keep `SWITCHES[0]` low to use the `sq_wave_gen` module to drive the DAC.

Use `make impl` and `make program` to **put the circuit on the FPGA and test it**.

Numerically Controlled Oscillator (NCO)

The top level schematic for the rest of this lab is shown below:



Overview

Now we can generate tunable square waves using `sq_wave_gen`, but 1) they sound harsh and 2) we want to create a more general wave generation circuit that has frequency control and supports arbitrary waveform types.

Let's use a numerically controlled oscillator (NCO) to generate sine waves. Here's the math involved:

A **continuous time** sine wave, with a frequency f_{sig} , can be written as:

$$f(t) = \sin(2\pi f_{sig} t)$$

If this sine wave is sampled with sampling frequency (so the audio could be stored as a bunch of numbers) f_{samp} ($= 125\text{e6} / 1024 = 122 \text{ kHz}$ in our case), the resulting stream of discrete time samples is:

$$f[n] = \sin(2\pi f_{sig} \frac{n}{f_{samp}})$$

We want to let our hardware output such stream of samples. One way to do this is to use a **lookup table (LUT)** and a **phase accumulator (PA)**, just a register that can increment its value).

Say we have a LUT that contains sampled points for one period of a sine wave with 2^N entries. The entries i (where $0 \leq i \leq 2^N - 1$) of this LUT are:

$$LUT[i] = \sin(i \frac{2\pi}{2^N})$$

To find the index i of the LUT that stores the **n -th sample**, we can equate the expressions inside `sin()`:

$$i \frac{2\pi}{2^N} = 2\pi f_{sig} \frac{n}{f_{samp}}$$

Which leads to the following:

$$i = \underbrace{\left(\frac{f_{sig}}{f_{samp}} 2^N \right)}_{\text{phase increment}} n$$

This means that to calculate sample $n+1$ for a given f_{sig} , we should take the LUT index i that corresponds to sample n and increment the index by the **frequency control word (FCW)** (also called the **phase increment** in the equation above).

To find the frequency step, Δf_{min} , of the NCO (a.k.a frequency resolution) we can look at how much of a change in f_{sig} could cause the FCW, or phase increment, to increase by 1:

$$\frac{f_{sig} + \Delta f_{min}}{f_{samp}} 2^N = \frac{f_{sig}}{f_{samp}} 2^N + 1$$

$$\Delta f_{min} = \frac{f_{samp}}{2^N}$$

In the equation above, 2^N is the total number of frequencies we could represent using N bits. In this lab we will use $N=24$. Recall that in lab 3, our DAC has a frequency of 122kHz, which means the frequency resolution is 0.007Hz. We can have very precise frequency control using an NCO.

However, a 2^{24} entry LUT is huge and wouldn't fit on the FPGA. So, we will keep the phase accumulator N (24-bits) wide, and only use the MSB M bits to index the sine wave LUT. This means the LUT only contains 2^M entries, where M can be chosen based on the tolerable phase error. **We will use $M = 8$ in this lab.**

To learn more about phase error and spurs that occur due to truncation refer to [this article from all about circuits](#), [this analog devices note](#), and [this MIT paper on phase truncation effects](#).

Implementation

We've generated a file that contains the contents of the LUT for you in `src/sine.bin`. You can run the following command to re-generate it:

```
python scripts/nco.py --sine-lut > sine.bin
```

We can use the file to initialize a ROM inside `src/nco.v`. Use the Single-port ROM with asynchronous read in the `EECS151.v` library file (`ASYNC_ROM` module). You can provide a file for it to load the ROM with initial contents like the following example (note how this uses `$readmemb()` under the hood).

```
ASYNC_ROM #(
    .DWIDTH(10), // Data width
    .AWIDTH(8),  // Address width
    .MIF_BIN("sine.bin") // Memory initialization file (binary)
) sine_rom (
    .q(/* output */),
    .addr(/* address */)
);
```

If you are running a simulation in GUI Vivado, you must add this file to sim sources.

Implement the NCO in `src/nco.v`. Note that the PA uses the main clock and runs at 125MHz. When `next_sample` is high, you should output a new DAC code on `out` on the next rising clock edge, similar to the `sq_wave_gen`. You can assume that `fcw` can change only when `next_sample` isn't high.

Verification

We have provided a testbench for the NCO in `sim/nco_tb.v`. It is similar to `sim/sq_wave_gen_tb.v` in that it uses one thread to dump to fetch samples from the NCO and dumps them to a file called `nco_codes.txt`, and it uses another thread to set `fcw`.

You can run the testbench as usual, with the provided assertions. You should also **modify the testbench** to produce a 440 Hz tone using the NCO by setting the correct `fcw`. You can use the same script to convert the sample outputs to an audio file.

```
../scripts/audio_from_sim sim/nco_codes.txt
aplay output.wav
```

Verify the simulated output sounds like a [pure sine tone at 440 Hz](#) rather than the harsh sound produced by a square wave generator.

FPGA

Look through `src/z1top.v` for the instantiation of the `nco`. Note that the `fcw` comes from an FSM which we will implement in the next part. Also note that `SWITCHES[0]` controls whether the square wave circuit (0) or the NCO (1) is playing through the audio jack and `SWITCHES[1]` can be used to mute the audio (0 = mute, 1 = active).

For now, hard-code `fcw` to the value required to play a 440 Hz tone.

```
// .fcw(fcw),
.fcw(24'd_____),
```

Run `make impl` and `make program`, and make sure you hear a 440 Hz sine wave when you plug in headphones to the audio jack.

FSM + Note Sequencer (RAM)

Sequencer RAM

We want to implement a sequencer that holds 4 notes (FCWs) and plays each note for 1 second through the NCO in a loop. We have provided a RAM that's used to hold and modify these 4 notes. See `src/fcw_ram.v` for a skeleton of a RAM with 1 read/write port. Note that both read and write are *synchronous*.

The RAM contains 4 24-bit values, which correspond to the FCWs for the 4 notes. Initially (upon reset) the RAM should hold these 4 notes:

- 440 Hz (A4)
- 494 Hz (B4)
- 523 Hz (C5)
- 587 Hz (D5)

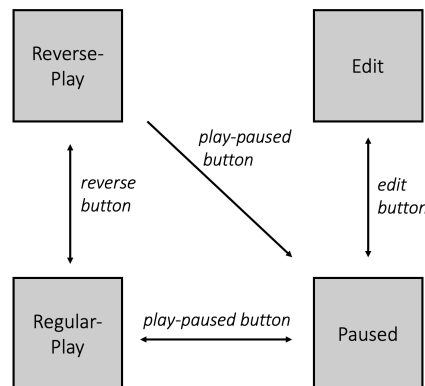
Calculate the corresponding FCWs and edit the define statements in `src/fcw_ram.v` with the values you calculated.

FSM Specification

With the sequencer RAM in place, we want to design and implement an FSM that will use the buttons to play, reverse-play, and pause the playback of the 4 notes in the sequencer RAM.

The FSM takes the lower 3 buttons as inputs and outputs the values for 4 LEDs and the FCW for the NCO. A skeleton is provided in `src/fsm.v`.

The FSM has 4 states: `REGULAR_PLAY`, `REVERSE_PLAY`, `PAUSED`, `EDIT`. Here is the state transition diagram:



- The initial state should be `REGULAR_PLAY`. In this state, the FSM should play the notes in the RAM one by one (440Hz -> 494Hz -> 523Hz -> 587Hz). Each note should be played for 1 second.
- Pressing the play-paused button (`button[0]`) should transition you into the `PAUSED` state from either the `REGULAR_PLAY` or `REVERSE_PLAY` states. Pressing the same button while in the `PAUSED` state should transition the FSM to the `REGULAR_PLAY` state.
- In the `PAUSED` state, the RAM address should be held steady at its value before the transition into `PAUSED`, and the NCO should freeze (e.g. set `fcw` to 0). After returning to the `REGULAR_PLAY` state, the RAM address should begin incrementing again from where it left off.
- You can toggle between the `REGULAR_PLAY` and `REVERSE_PLAY` states by using the reverse button (`button[1]`). In the `REVERSE_PLAY` state, you should decrement the RAM address by 1 rather than increment it by 1 every second.
- The `EDIT` state can only be entered when the edit button (`button[2]`) is pressed in the `PAUSED` state. In the `EDIT` state, the current note should come out of the speaker continuously. Pressing `button[0]` will decrease the frequency of the current tone, while pressing `button[1]` should increase the frequency. You can decide the step at will and it doesn't have to be linear. Pressing the edit button should transition the FSM back to the `PAUSED` stage.
- If the FSM is reset (`rst`) it should return to the `REGULAR_PLAY` state, and the RAM should be reset to its original values.
- If you don't press any buttons, the FSM shouldn't transition to another state.
- Keep in mind, when doing a reset, we cannot write the current note into the ram (what should `wr_en` look like?)
- **Make sure your note doesn't underflow or overflow** beyond the min/max frequency.

The `1eds` output should track which note you are playing (one-hot). The `1eds_state` output should represent the state your FSM is in using 2 bits.

We have provided a skeleton in `src/fsm.v` and a simple RAM in `src/fsm_ram.v`. If you would like to use a different implementation, feel free to modify it.

Here are some tips to create an FSM in verilog:

- Encode your states as local parameters (i.e. `localparam PLAY = 2'b01` at the top of the module) and use the labels instead of literal values (i.e. `next_state = PLAY`).
- If you are driving a value (such as outputs or internal signals) in `always @(*)` blocks, make sure to only drive that one value in a single block.
 - Not doing so will lead to multi-driven net errors in synthesis; tools will be unsure which driver (always block) should actually drive the wire.
 - `case` statements in the always blocks on the current state can be helpful.
- Recall that any value assigned to in an `always @(*)` block must be declared as a `reg`.

- Note this value still represents combinational logic - `reg wr_en;` can only ever synthesize to a register / represent sequential logic if assigned to in an `always @(posedge clk)` block, but this infers a flip flop which is **not allowed** in this course.

Testbench

We have provided an FSM testbench skeleton in `sim/fsm_tb.v`.

You should **edit it** to simulate pressing buttons and verifying the FSM behaves correctly. Make sure you test all the state transitions.

You can override `CYCLES_PER_SECOND` when instantiating `fsm` to speed up simulation (as before).

Put Everything Together

Check the top-level diagram again. Make sure that all modules are connected as desired in `src/z1top.v` (remove the `fcw` hardcoding from the previous part).

Program the FPGA. Plug headphones into the audio out port, press the reset button, and verify that you hear the notes in the sequencer RAM play one after the other. Use the buttons to switch between different states.

Lab Deliverables

Lab Checkoff (due: next lab)

To checkoff for this lab, have these things ready to show the TA:

- Demonstrate the tunable square wave generator on the FPGA (reset, linear/exponential frequency adjustment)
- Demonstrate the FSM on the FPGA (FSM with play, reverse play, pause, edit, and reset)

Acknowledgement

This lab is the result of the work of many EECS151/251 GSIs over the years including:

- Sp12: James Parker, Daiwei Li, Shaoyi Cheng
- Sp13: Shaoyi Cheng, Vincent Lee
- Fa14: Simon Scott, Ian Juch
- Fa15: James Martin
- Fa16: Vighnesh Iyer
- Fa17: George Alexandrov, Vighnesh Iyer, Nathan Narevsky
- Sp18: Arya Reais-Parsi, Taehwan Kim
- Fa18: Ali Moin, George Alexandrov, Andy Zhou
- Sp19: Christopher Yarp, Arya Reais-Parsi
- Fa19: Vighnesh Iyer, Rebekah Zhao, Ryan Kaveh
- Sp20: Tan Nguyen
- Fa20: Charles Hong, Kareem Ahmad, Zhenghan Lin
- Sp21: Sean Huang, Tan Nguyen
- Fa21: Charles Hong, Vighnesh Iyer, Alisha Menon, Zhenghan Lin
- Sp22: Alisha Menon, Yikuan Chen, Seah Kim
- Fa22: Paul Kwon, Yikuan Chen
- Fa23: Viansa Schmulbach
- Sp24: Daniel Endraws

EECS150 / fpga_labs_sp24 Public

<> Code Issues Pull requests Actions Projects Security Insights

fpga_labs_sp24 / lab5 /

 daniel-endraws minor lab updates ab4190a · last year

Name	Name	Last commit date
..		
figs	add lab5	last year
sim	add lab5	last year
src	add lab5	last year
Makefile	add lab5	last year
README.md	minor lab updates	last year

README.md

FPGA Lab 5: UART and FIFO

fpga_labs_sp24 / lab5 / ↑ Top

Prof. John Wawryznek

TAs: (ordered by section) Daniel Endraws, Dhruv Vaish, Rohit Kanagal

Department of Electrical Engineering and Computer Science

College of Engineering, University of California, Berkeley

Overview

In this lab we will:

- Understand the ready-valid interface
- Design a universal asynchronous receiver/transmitter (UART) circuit
- Design a first-in-first-out (FIFO) circuit

There is no FPGA testing required for this part.

Before You Begin

Ensure you have committed and pushed previous changes to your github classroom, then fetch the latest lab skeleton

```
cd fpga-labs-<username>
git pull staff main
```

If you face any divergent branch issues, merge the staff changes into your branch with `git merge staff/main`

Copy the following sources from the previous lab

- lab5/src/debouncer.v
- lab5/src/synchronizer.v
- lab5/src/edge_detector.v

Remember that you must not use any `always @(posedge clk)` blocks in modules intended for synthesis. Use the registers in the `EECS151.v` library instead. Using `always @(posedge clk)` in testbenches is fine, since they are not synthesizable.

Reading

- Read this document on [ready-valid interfaces](#)

Debugging

If your Verilog isn't working as expected, here are some things you should check:

- Does `make lint` report any errors/warnings (besides the `UNOPTFLAT` issue)?
- Are you using any `posedge clk` blocks? You should only be using combinational always blocks.

- Are all your registers instantiations of the ones in the `EECS151.v` library?
- Are there any inferred latches in the synthesis logs? Any warnings about multi-driven nets? If a variable is assigned in one combinational logic branch, remember that it must be assigned in **all** branches.
- Have you looked at the waveforms for relevant signals? Look for off-by-one-cycle timing issues and Xs (red waveforms).
- Does your code use non-synthesizable constructs? Ensure you are not using system functions, delays, or X's/Z's in modules you intend to synthesize.

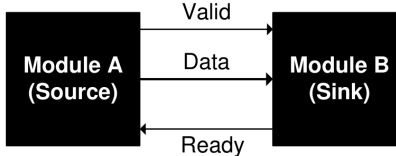
▼ Table of Contents (click to close)

- [FPGA Lab 5: UART and FIFO](#)
 - [Overview](#)
 - [Before You Begin](#)
 - [Reading](#)
 - [Debugging](#)
 - [Ready-Valid Interfaces](#)
 - [UART Serial Device](#)
 - [UART Packet Framing](#)
 - [UART Receiver](#)
 - [UART Transmitter](#)
 - [UART Transmitter Verification](#)
 - [FIFO](#)
 - [FIFO Functionality](#)
 - [FIFO Interface](#)
 - [FIFO Timing](#)
 - [FIFO Testing](#)
 - [FIFO Verification \(Optional\)](#)
 - [Lab Deliverables](#)
 - [Acknowledgement](#)

Ready-Valid Interfaces

Often, we want to design modules that pass data between each other but are unaware of each other's internal timing. The *ready-valid interface* is a standardized interface and protocol for timing-agnostic unidirectional data transfer between 2 modules that share a clock.

The ready-valid interface is used to send data from a *source* to a *sink*.



It consists of 3 wires:

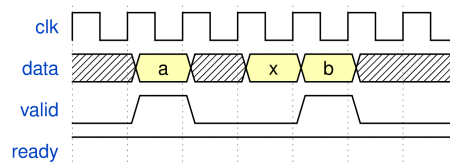
- `valid` (1 bit, driven by source, received by sink)
- `data` (D bits, driven by source, received by sink)
- `ready` (1 bit, driven by sink, received by source)

The sink uses the `ready` signal to tell the source if the sink is able to receive a new piece of data. Usually the sink can only take in a certain number of data pieces until it is full and has to process what it has already received before it can receive any more.

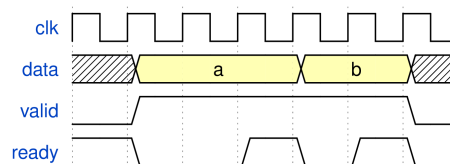
The source uses the `valid` signal to tell the sink that the `data` bus has data it wishes to send to the sink.

A ready-valid *transaction* only occurs when both `ready` and `valid` are high on a rising clock edge. If both `ready` and `valid` are high on a rising edge, the source can assume that the sink has received and *internally stored* the bits on `data`.

Here are a few examples:



There are two transactions that take place on the 3rd and 6th rising clock edges. Note that the source can change `data` when `valid` is low.



The sink can pull `ready` low whenever it isn't ready to accept new data. In this example, there are 2 transactions that take place on the 5th and 7th rising clock edges. When the source has `valid` high, but the sink has `ready` low we say that the sink is applying *backpressure* to the source.

The data transfer from source to sink only occurs when *both* `ready` and `valid` are high on a rising edge.

UART Serial Device

In this lab, we will design a circuit that implements the UART serial protocol for transmitting and receiving data over a serial interface. This will enable circuits on the FPGA to communicate with the workstation, which will allow us to programmatically send data to and from the FPGA.

UART is a 2 wire protocol with one wire carrying data from the workstation to the FPGA and the other one carrying data from the FPGA to the workstation. Here is an overview of the setup we will use:

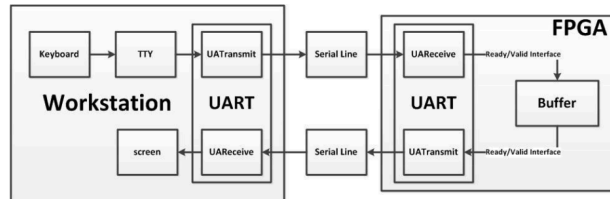


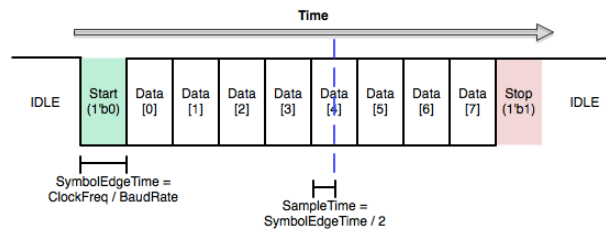
Diagram of the entire setup

The UART transmit and receive modules use a *ready-valid interface* to communicate with other modules on the FPGA. Both the UART's receive and transmit modules will have their own separate ready-valid interface connected appropriately to other modules.

Please note that the serial line itself is not a ready-valid interface. Rather, it is the modules you will work with in this lab (`uart_transmitter` and `uart_receiver`) that use the ready-valid handshake.

UART Packet Framing

On the `PYNQ-Z1` board, the physical signaling aspects (such as voltage level) of the serial connection will be taken care of by off-FPGA devices. From the FPGA's perspective, there are two signals, `FPGA_SERIAL_RX` and `FPGA_SERIAL_TX`, which correspond to the receive-side and transmit-side pins of the serial port. The FPGA's job is to correctly frame 8-bit data words going back and forth across the serial connection. The figure below shows how a single 8-bit data word is transferred over the serial line using the UART protocol.

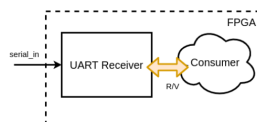


Framing of a UART packet

In the idle state the serial line is held high. When the TX side is ready to send an 8-bit word, it pulls the line low. This is called the start bit. Because UART is an asynchronous protocol, all timing within the frame is relative to when the start bit is first sent (or detected, on the receive side).

The frame is divided into 10 uniformly sized bits: the start bit, 8 data bits, and then the stop bit. The width of a bit in cycles of the system clock is given by the system clock frequency (`125 MHz`) divided by the baudrate. The baudrate is the number of bits sent per second; in this lab the baudrate will be `115200`. Notice that both sides must agree on a baudrate for this scheme to be feasible.

UART Receiver



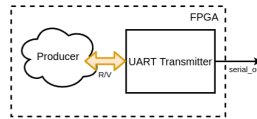
Connectivity of the UART receiver

The receive side of the UART is just a shift register that shifts bits in from the serial line. However, care must be taken into determining *when* to shift bits in. If we attempt to sample the `FPGA_SERIAL_RX` signal directly on the edge between two symbols, we are likely to sample on the wrong side of the edge and get the wrong value for that bit. One solution is to wait halfway into a cycle (until `sampleTime` on the diagram) before reading a bit in to the shift register.

The UART receiver module sends the received 8-bit word to a consumer block on the FPGA via a ready-valid interface. Once we have received a full UART packet over the serial port, the `valid` signal should go high until the `ready` signal goes high, after which the `valid` signal will be driven low until we receive another UART packet.

You do not need to implement the UART receiver as it is provided to you in `lab5/src/uart_receiver.v`, but you should refer to its implementation when writing the `uart_transmitter`.

UART Transmitter



Connectivity of the UART transmitter

The UART Transmitter receives an 8-bit word from a producer block on FPGA via the ready-valid interface. Once we have a 8-bit word that we want to send (i.e., once `valid` is high, and the transmitter is `ready`), transmitting it involves shifting each bit of the `data[7:0]` bus, plus the start and stop bits, out of a shift register on to the serial line.

Remember, the serial baudrate is much slower than the system clock, so we must wait $\text{SymbolEdgeTime} = \text{ClockFreq} / \text{BaudRate}$ cycles between changing the bit we're putting on the serial line. After we have shifted all 10 bits out of the shift register, we are done unless we have to send another frame immediately after. The transmitter should not be `ready` when it is in a middle of sending a frame.

Your task is to complete the implementation of the UART transmitter in `lab5/src/uart_transmitter.v`.

UART Transmitter Verification

We have provided 2 testbenches to verify the UART transmitter.

- `sim/uart_transmitter_tb.v`
- `sim/uart2uart_tb.v`

You can run them as usual; they will print out any errors during execution.

FIFO

A FIFO (first in, first out) data buffer is a circuit that allows data elements to be queued through a write interface, and read out sequentially by a read interface. The FIFO we will build in this section will have both the read and write interfaces clocked by the same clock; this circuit is known as a synchronous FIFO.

FIFO Functionality

A FIFO is implemented with a circular buffer and two pointers: a read pointer and a write pointer. These pointers address the buffer inside the FIFO, and they indicate where the next read or write operation should be performed. When the FIFO is reset, these pointers are set to the same value.

When a write to the FIFO is performed, the write pointer increments and the data provided to the FIFO is written to the buffer. When a read from the FIFO is performed, the read pointer increments, and the data present at the read pointer's location is sent out of the FIFO.

When the read pointer equals the write pointer, the FIFO is either full or empty. The implementation of the FIFO logic is up to you, but you may wish to store one extra bit of state to distinguish these two conditions. The `Electronics` section of the [FIFO Wikipedia article](#) will likely aid you in creating your FIFO.

Here is a block diagram of a FIFO similar to the one you should create, from page 103 of the [Xilinx FIFO IP Manual](#).

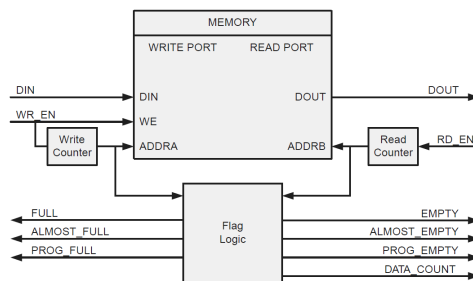


Figure 5-4: Functional Implementation of a Common Clock FIFO using Block RAM or Distributed RAM

The interface of our FIFO will contain a *subset* of the signals enumerated in the diagram above.

FIFO Interface

Look at the FIFO skeleton in `src/fifo.v`.

The FIFO is parameterized by:

- `WIDTH` - The number of bits per entry in the FIFO
- `DEPTH` - The number of entries in the FIFO.
- `POINTER_WIDTH` - The width of the read and write pointers.

The common FIFO signals are:

- `clk` - Clock used for both read and write interfaces of the FIFO.
- `rst` - Reset (synchronous with `clk`); should force the FIFO to become empty.

The FIFO write interface consists of:

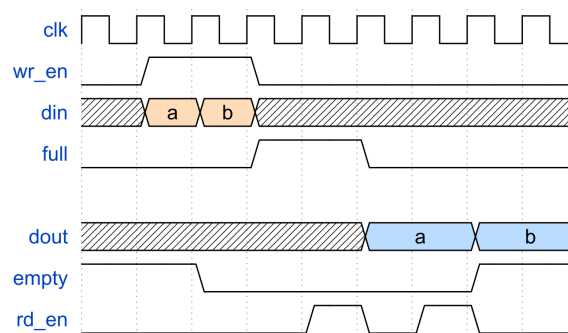
- input `wr_en` - When this signal is high, on the rising edge of the clock, the data on `din` should be written to the FIFO.
- input `[WIDTH-1:0] din` - The data to be written to the FIFO.
- output `full` - When this signal is high, the FIFO is full. When the FIFO is full, you should not accept any new data, even if `wr_en` is high.

The FIFO read interface consists of:

- input `rd_en` - When this signal is high, on the rising edge of the clock, the FIFO should output the data indexed by the read pointer on `dout`.
- output `[WIDTH-1:0] dout` - The data that was read from the FIFO after the rising edge on which `rd_en` was asserted.
- output `empty` - When this signal is high, the FIFO is empty. Attempting to read from an empty FIFO (by raising `rd_en`) should not corrupt any of the internal state of your FIFO.

FIFO Timing

The FIFO that you design should conform to the specs above. Here is a timing diagram for a 2-entry FIFO. Note that the data on `dout` only changes *after the rising edge* when `rd_en` is high.



FIFO Testing

We have provided a testbench in `sim/fifo_tb.v`.

The testbench performs the following test sequence:

- Checks initial conditions after reset (FIFO not full and is empty)
- Generates random data which will be used for testing
- Pushes the data into the FIFO, and checks at every step that the FIFO is no longer empty
- When the last piece of data has been pushed into the FIFO, it checks that the FIFO is not empty and is full
- Verifies that cycling the clock and trying to overflow the FIFO doesn't cause any corruption of data or corruption of the full and empty flags
- Reads the data from the FIFO, and checks at every step that the FIFO is no longer full
- When the last piece of data has been read from the FIFO, it checks that the FIFO is not full and is empty
- Verifies that cycling the clock and trying to underflow the FIFO doesn't cause any corruption of data or corruption of the full and empty flags
- Checks that the data read from the FIFO matches the data that was originally written to the FIFO
- Prints out test debug info

This testbench tests particular ways of interfacing with the FIFO. Of course, it is not comprehensive, and there are conditions and access patterns it does not test. As writing testbenches is a very valuable skill, we **highly recommend** reading through the testbench to understand what it does and adding some more tests to this testbench (or writing a new testbench) to verify your FIFO performs as expected.

FIFO Verification (Optional)

To more powerfully verify the FIFO in a much more concise manner, we can implement a few basic SystemVerilog Assertions (SVA). Read/skim through [this tutorial](#), then try to implement properties/assertions to verify the following **concurrently**:

1. When the FIFO is full, the write pointer does not change unless reset is asserted.
2. When the FIFO is empty, the read pointer does not change unless reset is asserted.
3. When reset is asserted, the read and write pointers are set to 0, and full is deasserted.

The assertions you write can be placed in `fifo.v`.

It is possible to write more sophisticated assertions to check, for example, that the data you read from the FIFO equals the data you wrote to it.

Lab Deliverables

Lab Checkoff (due: next lab)

Once you have finished implementing and testing the UART and FIFO, you are done with lab 5!

Commit and push your code to GitHub. Check that you are not pushing generated files (such as waveform dumps). Do commit the Verilog source for any testbenches you write/modify.

To checkoff for this lab, have these things ready to show the TA:

- Go through the UART simulation results and show that your UART behaves as expected. What do the testbenches do?
- Go through the FIFO simulation results and show it works correctly. You are not required to program your design on an FPGA for Lab 5 (in Lab 6, we'll use your FIFO and UART from Lab 5).

Acknowledgement

This lab is the result of the work of many EECS151/251 GSIs over the years including:

- Sp12: James Parker, Daiwei Li, Shaoyi Cheng
- Sp13: Shaoyi Cheng, Vincent Lee
- Fa14: Simon Scott, Ian Juch
- Fa15: James Martin
- Fa16: Vighnesh Iyer
- Fa17: George Alexandrov, Vighnesh Iyer, Nathan Narevsky
- Sp18: Arya Reais-Parsi, Taehwan Kim
- Fa18: Ali Moin, George Alexandrov, Andy Zhou
- Sp19: Christopher Yarp, Arya Reais-Parsi
- Fa19: Vighnesh Iyer, Rebekah Zhao, Ryan Kaveh
- Sp20: Tan Nguyen
- Fa20: Charles Hong, Kareem Ahmad, Zhenghan Lin
- Sp21: Sean Huang, Tan Nguyen
- Fa21: Vighnesh Iyer, Charles Hong, Zhenghan Lin, Alisha Menon
- Sp22: Alisha Menon, Yikuan Chen, Seah Kim
- Fa22: Yikuan Chen, Raghav Gupta, Ella Schwarz, Paul Kwon, Jennifer Zhou
- Sp23: Rahul Kumar, Yukio Miyasaka, Dhruv Vaish
- Fa23: Nikhil Jha
- Sp24: Daniel Endraws

EECS150 / fpga_labs_sp24 Public

<> Code Issues Pull requests Actions Projects Security Insights

fpga_labs_sp24 / lab6 / ...

 daniel-endraws [release lab6](#)

8eac9f · last year 

Name	Name	Last commit date
..		
figs	release lab6	last year
sim	release lab6	last year
src	release lab6	last year
Makefile	release lab6	last year
README.md	release lab6	last year

README.md ⋮

FPGA Lab 6: Memory Controller

Prof. John Wawryznek

TAs: (ordered by section) Daniel Endraws, Dhruv Vaish, Rohit Kanagal

Department of Electrical Engineering and Computer Science

College of Engineering, University of California, Berkeley

Overview

In this lab we will:

- Connect the FIFO and UART circuits together, bridging two ready-valid interfaces.
- Design a memory controller that takes read and write commands from a FIFO, interacts with a synchronous memory accordingly, and returns read results over another FIFO.
- Optional - build a fixed length piano.

This part will involve testing your design on FPGA.

Before You Begin

Ensure you have committed and pushed previous changes to your github classroom, then fetch the latest lab skeleton

```
cd fpga-labs-<username>
git pull staff main
```

If you face any divergent branch issues, merge the staff changes into your branch with `git merge staff/main` , fixing any merge conflicts.

Also copy the following sources from previous labs:

```
cp lab4/src/synchronizer.v lab4/src/debouncer.v lab4/src/edge_detector.v lab6/src
cp lab5/src/fifo.v lab5/src/uart_transmitter.v lab6/src
```

Remember that you must not use any `always @(posedge clk)` blocks in modules intended for synthesis. Use the registers in the `EECS151.v` library instead. Using `always @(posedge clk)` in testbenches is fine, since they are not synthesizable.

▼ Table of Contents (click to close)

- [FPGA Lab 6: Memory Controller](#)
 - [Overview](#)
 - [Before You Begin](#)
 - [Memory Controller](#)
 - [Read/Write Packet](#)
 - [Operation of the Memory Controller](#)
 - [Running the testbench](#)
 - [On the FPGA](#)
 - [PMOD USB-UART](#)

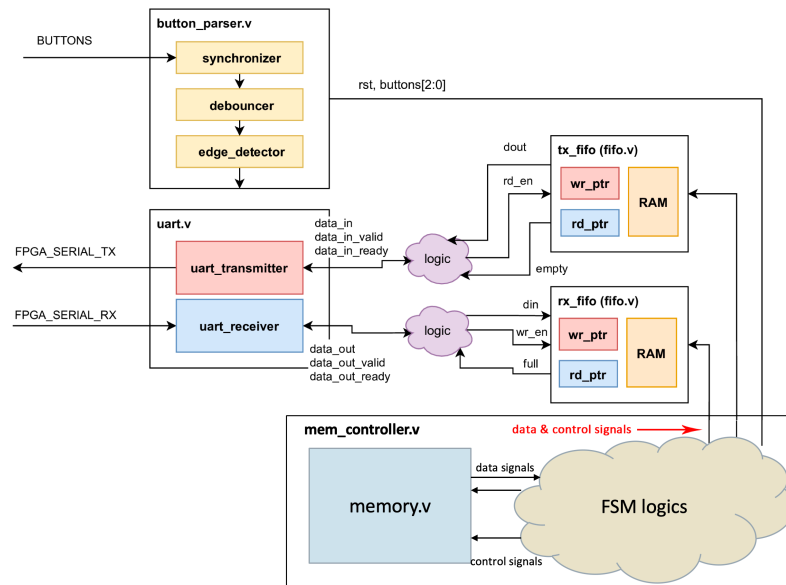
https://github.com/EECS150/fpga_labs_sp24/tree/main/lab6

1/6

- [Hello World](#)
- [Lab Deliverables](#)
- [Optional - Fixed Note Length Piano](#)
 - [Modify z1top](#)
 - [Writing a System-Level Testbench](#)
 - [FPGA Testing](#)
- [Acknowledgement](#)

Memory Controller

One of the key enabling blocks for the RISC-V CPU on the Final Project of this course is Memory-Mapped I/O. Specifically, we will use UART that we build in this lab to interface between a host computer and a synchronous memory block. An instance of its use is to write instructions into the instruction memory with the UART interface so our CPU can run those instructions. In this lab, we will build a simple UART-FIFO-MEMORY interface to get you familiarized with working with RAMs.



Block diagram of the system, note that the connections here are just for the purpose of illustration and do not represent all connections

Read/Write Packet

The host side (your workstation computer) will send a two-byte packet (for read operation) or a three-byte packet (for write operation) to the FPGA via UART.

Read operation packet: {READ , ADDR} READ: 8'd48

Write operation packet: {WRITE, ADDR, DATA} WRITE: 8'd49

Format of data packets, write (ASCII '1' = 8'd49) is 1 on the keyboard, and read (ASCII '0' = 8'd48) is 0 on the keyboard

Operation of the Memory Controller

The role of the memory controller is to handle memory reads and writes based on commands that the user sends from the host computer. Each operation is a multi-cycle process which consists of different states. You do not need to support simultaneous reads and writes.

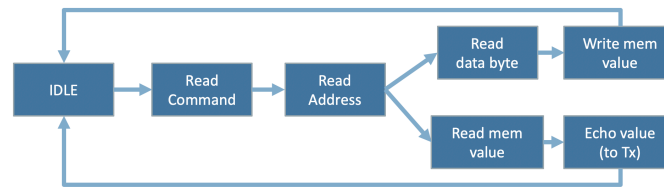
This module uses a `SYNC_RAM_WBE` from the `EECS151.v` library, which is a synchronous read RAM with a write byte enable signal. In order to handle memory reads and writes, your FSM needs to set the `mem_din` (memory input data), `mem_addr` (memory address), `mem_we` (memory write enable) and FIFO communication signals properly.

Commands are either 2 bytes long (for reads), or 3 bytes long (for writes). The first byte indicates whether you should perform a read or a write. The second byte is the address to read from or write to. For write commands, the third byte is the data to write into memory.

1. Your memory controller FSM should start with an IDLE state upon pressing the reset button. It should make a read (the 1st byte) from the RX FIFO whenever the `rx_fifo_empty` signal becomes 0. Then, it should wait for the next packet (the address byte) to arrive at the FIFO, so it can read that byte (the 2nd byte).
 - *Extremely Important Tip:* Refer to the FIFO timing diagram to see when the FIFO's output data is valid! Remember we're using a synchronous FIFO.
2. Next, depending on whether a Read (8'd48, or key 0 on the keyboard) or a Write (8'd49, or key 1 on the keyboard) command has been received for the first byte, the memory controller transitions into different states.

- If the command was `Write`, then the FSM should wait for the data byte (**the 3rd byte**) to become available in the FIFO. Once the data byte is read, the controller should write the data byte into the correct address in memory, then return to IDLE.
- If the command was `Read`, then the value at the corresponding address should be read from the RAM, and then sent to the TX_FIFO (setting control signals accordingly), followed by returning to the IDLE state. Again, remember the implications that a synchronous FIFO has for the timing of your control signals.

You can use `state_leds` to monitor current state on the FPGA. Some states will pass too quickly to be visible, but the "IDLE" and input states involve waiting and hence should be easily visible. We will not be testing you on the use of `state_leds`.



Flow diagram of the FSM. Note that transitions back to the current state are not shown here

Keep in mind that the input bytes might not be sent back-to-back, so your FSM has to wait in the current state until it receives the next byte.

We have provided you with a skeleton for the FSM in `src/mem_controller.v`. You will see it is structured like so:

- State update using `state_reg` register
- Next state logic (`always @(*)`)
- Output and memory signal logic (`always @(*)`)
- Space for registers for byte reading & packet counting, if needed

Please feel free to use a different number of states, or different states, as long as you completely understand the required behavior and the design of your FSM. However, you should not change the controller interface.

In some FSM implementations, you could see people combining 2 & 3 into the same `always@(*)` block, it is just personal preference.

Running the testbench

Once you finish `mem_controller.v` run the `sim/mem_controller_tb.v` testbench.

If the simulation doesn't finish (gets stuck), press `ctrl+c` and type `quit`, then open up the `dve` tool to check the waveform. Does the timing of each state transition and control signal look correct (refer to the FIFO timing diagram from lab 5)?

If you see all tests passed, proceed to running the system level testbench `sim/system_tb.v`, which requires your top level, uart, fifo, and memory controller to all work together.

Note that both `mem_controller_tb.v` and `system_tb.v` require a correct FIFO to interface with the memory controller.

If everything looks correct, program your FPGA (see next section).

On the FPGA

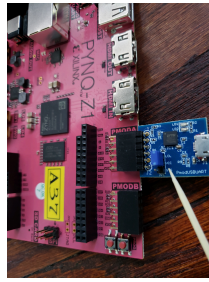
Use the standard `make impl` and `make program` to create and program a bitstream.

Pay attention to the warnings generated by Vivado in `build/synth/synth.log`. It's possible to write your Verilog in such a way that it passes behavioural simulation but doesn't work in implementation. Warnings about `multi driven nets`, for example, can lead to certain logic pathways being optimized out. Latch synthesis is another notable cause of mismatch between simulation and FPGA behavior.

PMOD USB-UART

The PYNQ-Z1 does not have an RS-232 serial interface connected to the FPGA fabric. So we'll be using the [PMOD USB-UART](#) extension module to add a UART interface to the Pynq board. Connect the PMOD module to the **top** row of the PMOD A port on the Pynq, and connect a USB cable from the USB-UART PMOD to your computer (this is already done in the Cory 111 workstations).

Note: Make sure that the power selection jumper on the PMOD USB-UART is set to LCL3V3.



PMOD USB-UART plugged in with correct power jumper setting (blue).

Hello World

Make sure `SWITCH[0]` is at "off(0)" position so you are in the memory controller mode. Reset the UART circuit on your FPGA with `buttons[0]`.

On your workstation, run:

```
screen $SERIALTTY 115200
```



This opens `screen`, a terminal emulator, connected to the serial device with a baud rate of 115200. When you type a character into the terminal, it is sent to the FPGA over the `FPGA_SERIAL_RX` line, encoded in ASCII. When the memory controller sends a new character, it will be pushed over the `FPGA_SERIAL_TX` line to your workstation computer. When `screen` receives a character, it will display it in the terminal.

To test your implementation, type one character at a time. Send write packets (remember the byte corresponding to each character will be its ASCII value; ASCII charts are readily available online). Then send read packets with addresses you've written to and ensure you receive the data written earlier.

If you see some weird garbage symbols then the data is getting corrupted and something is likely wrong. If you see this happening very infrequently, don't just hope that it won't happen while the TA is doing the checkoff; take the time now to figure out what is wrong. UART bugs are a common source of headaches for groups during the final project.

To close `screen`, type `Ctrl-a` then `Shift-k` and answer `y` to the confirmation prompt. If you don't close `screen` properly, other students won't be able to access the serial port on your workstation.

If you try opening `screen` and it terminates after a few seconds with an error saying `Sorry, can't find a PTY or Device is busy`, execute the command `killscreen` which will kill all open `screen` sessions that other students may have left open. Then run `screen` again.

Use `screen -r` to re-attach to a non-terminated `screen` session. You can also reboot the computer to clear all active `screen` sessions.

Lab Deliverables

Lab Checkoff (due: next lab)

To checkoff for this lab, have these things ready to show the TA:

- Demonstrate on FPGA that it can perform write and read operations with the memory: 1. read after write, in random addresses and orders (e.g. W-R-W-R or W-W-R-R etc.)

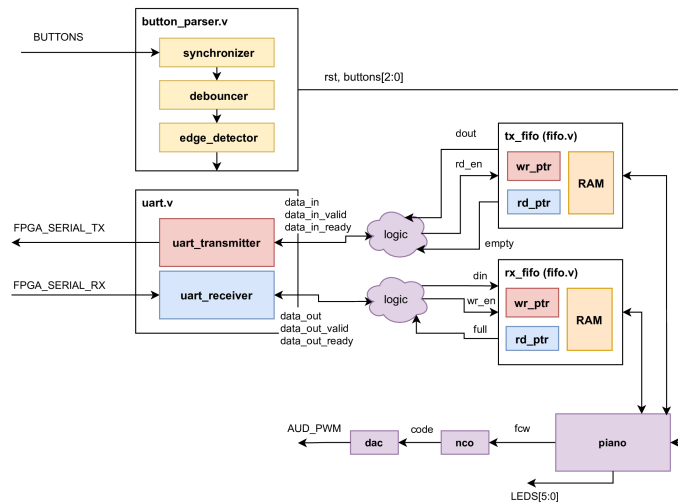
Optional - Fixed Note Length Piano

First copy sources from lab 4:

```
cp lab4/src/nco.v lab4/src/dac.v lab6/src
```



The piano interfaces the UART with the NCO + DAC. You should be able to switch into this mode (from the `memory_controller` mode) by switch `SWITCH[0]` to `on(1)`. Its job is to fetch notes sent from the UART, convert them to a `fcw`, and send them to the `nco` (and `dac`) for a **fixed amount of time**.



Block diagram of the system with fixed-length piano

Implement `src/fixed_length_piano.v`. It has access to the UART transmitter FIFO, the UART receiver FIFO, and the `nco` inputs. It also has access to a reset signal and the other buttons.

The piano should implement the following functionality:

- When the UART receiver FIFO contains a character, the FSM should pull the character from the FIFO and echo it back without modification through the UART transmitter FIFO.
- Once a character is pulled, its corresponding `fcw` should be read from the supplied `piano_scale_rom.v`.
- For a fixed amount of time (`note_length`), the note should be played by sending it to the `nco`. When a note isn't being played, the `fcw` should be set to 0
 - The `note_length` should default to 1/5th of a second, and can be changed by a fixed amount with the buttons.
 - `buttons[0]` increases the `note_length` and `buttons[1]` decreases the `note_length`
- Through doing all of this, your FSM should take care to ensure that if a FIFO is full, that it waits until it isn't full before pushing through data.
- You can use the `leds` output for debugging

You don't need to design the `fixed_length_piano` as an explicit FSM with states; the design is entirely up to you.

A ROM containing mappings from ASCII character codes to the `fcw` of the note to be played can be found in `src/piano_scale_rom.v`.

If you wish to re-generate this file, use these commands:

```
cd lab6
../scripts/piano_scale_generator scale.txt
../scripts/rom_generator scale.txt src/piano_scale_rom.v 256 24
# Modify piano_scale_rom.v to change the module name to 'piano_scale_rom'
```



It is possible that the UART receiver FIFO can fill up with samples so fast that the piano can't keep up; similar overflow conditions are possible with other parts of this system. You don't need to concern yourself with detecting 'backpressure' on the entire system and can just assume that your FIFOs are large enough to buffer all the user input.

Modify z1top

Open up `z1top.v` and modify it to include the new modules you wrote by uncommenting the last section of the file. Wire up the FIFOs and your `fixed_length_piano` + `nco` + `dac` according to the block diagram in the lab intro.

Make sure that you parameterize your FIFOs properly so that they have the proper `WIDTH` and `DEPTH`. You can make your FIFOs as deep as you want, but 8 should be enough.

Writing a System-Level Testbench

This design involves many moving parts that communicate with each other. We want to make sure that the complete integration of our system works as expected. To that end, you will have to write a system-level testbench that stimulates the top-level of your design and observes the top-level outputs to confirm correct behavior.

We have provided a template for a system testbench in `sim/piano_system_tb.v`. Fill in the `initial` block to test all the parts of the piano.

To make the waveform shorter and easier to debug, you can change the `CYCLES_PER_SECOND` parameter when instantiating `z1top` (and your `fixed_length_piano`) to something much shorter than 1/5th of a second. You will want to check that the `fcw` being sent to the NCO is correct; you should use hierarchical paths and asserts to do this:

```
assert(top.nco.fcw == 24'dxxxxxx);
```



Note, you can get the integer corresponding to a ASCII character in Python (and vice versa):

```
>>> ord('z')
122
>>> chr(122)
'z'
```



You will also want to check that the characters being sent to `z1top` are being echoed back correctly. Use the `off_chip_uart`'s `data_out`, `data_out_valid`, `data_out_ready` interface to do this.

FPGA Testing

Generate a bitstream and program the FPGA as usual. Read the synthesis and implementation reports (`build/synth/synth.log`) to see if there are any unexpected warnings.

You should watch out specifically for warnings like `found x-bit latch` or `x signal unconnected` or `x signal assigned but never used`. If you see that the synthesis tool inferred a latch, you should definitely fix that warning by completing any if-elseif-else or case statements that don't have a default signal assignment. The other 2 warning types are dependent on your design and you should ignore them only if you know they are expected.

Once you put your design on the FPGA you can send data to the on-chip UART by using `screen $SERIALTTY 115200`. The piano keys are mapped such that `z` through `<` go from C3 to C4 (middle C) and `q` through `i` go from C4 to C5. Holding down shift moves the lower row's octave down and the upper row's octave up.

Use `buttons[0]` to reset the design. Use `buttons[1]` and `buttons[2]` to change the `note_length` of your piano.

You should test the case where you make the `note_length` long, and fill up your UART FIFO by typing really fast. Then watch your FIFO drain slowly as each note is played for `note_length` time.

Acknowledgement

This lab is the result of the work of many EECS151/251 GSIs over the years including:

- Sp12: James Parker, Daiwei Li, Shaoyi Cheng
- Sp13: Shaoyi Cheng, Vincent Lee
- Fa14: Simon Scott, Ian Juch
- Fa15: James Martin
- Fa16: Vighnesh Iyer
- Fa17: George Alexandrov, Vighnesh Iyer, Nathan Narevsky
- Sp18: Arya Reais-Parsi, Taehwan Kim
- Fa18: Ali Moin, George Alexandrov, Andy Zhou
- Sp19: Christopher Yarp, Arya Reais-Parsi
- Fa19: Vighnesh Iyer, Rebekah Zhao, Ryan Kaveh
- Sp20: Tan Nguyen
- Fa20: Charles Hong, Kareem Ahmad, Zhenghan Lin
- Sp21: Sean Huang, Tan Nguyen
- Fa21: Vighnesh Iyer, Charles Hong, Zhenghan Lin, Alisha Menon
- Sp22: Alisha Menon, Yikuan Chen, Seah Kim
- Fa22: Yikuan Chen, Raghav Gupta, Ella Schwarz, Paul Kwon, Jennifer Zhou
- Sp23: Rahul Kumar, Yukio Miyasaka, Dhruv Vaish
- Fa23: Lucy Meng
- Sp24: Daniel Endraws